salaries, Cedar returns `deny`, and the request is blocked before it reaches the HR API.

This gateway enforcement pattern provided ACME with two key advantages. First, application developers, like the Manager Dashboard team, no longer needed to duplicate authorization logic; they simply passed principal information and relied on the APIs to enforce policies consistently. Second, authorization at the gateway protected ACME from coding errors or malicious actions. Even if the dashboard UI showed actions not intended for the current employee or if someone tried to tamper with the interface, the gateway would block unauthorized requests before they accessed sensitive data.

By making the gateway the universal PEP, ACME ensured that Cedar policies protected every call in the same way, whether from the Manager Dashboard or any other system.

11.1.2 Pattern 2: Service mesh with sidecar enforcement

The API gateway solved many problems, but Bob realized it couldn’t handle everything. Within ACME’s infrastructure, services constantly communicated with each other. For instance, the Manager Dashboard didn’t just query the HR API; it also retrieved budget data from Finance and project details from the Customer Collaboration platform. These requests often triggered chains of downstream calls. The HR API might call a directory service, or the Collaboration API might fan out to the Search service to locate documents. Much of this *east–west traffic* never passed through the edge gateway.